Scenario Name

Scenario 1 — Search Functionality

Weak Prompt

"Test the search feature."

What I Feel is Missing

No product type (e-commerce, hospital), no filter/sort behavior, no empty/invalid input scenarios, no performance angle.

1. Instruction

Act as a Senior QA Automation Engineer with expertise in Selenium, TestNG, Java, and web application testing. Analyze the Google Search functionality and generate comprehensive functional, negative, boundary, usability, and accessibility test scenarios.

2. Context

The application under test is the Google Search homepage. The primary objective is to validate that users can search using different types of input and receive accurate, relevant, and reliable search results. The testing should cover happy paths, edge cases, error handling, browser compatibility, responsiveness, and performance-related considerations.

3. Example

For example:

Scenario: User searches for "Selenium WebDriver".

Expected Result:

- Google accepts the input.
- Search results page loads successfully.
- Relevant results related to Selenium WebDriver are displayed.
- The search keyword appears in the search box.
- No unexpected errors occur.

4. Persona

You are a **Senior QA Engineer** with over 10 years of experience in web application testing.

5. Output Format

Provide the output in a table with the following columns:

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Priority	Test Type

Also include:

- Positive test scenarios
- Negative test scenarios
- Boundary test cases
- Special character test cases
- Cross-browser scenarios
- Automation feasibility (Yes/No)

6. Tone

Use a professional, structured, and QA-focused tone.

✅ Rewritten Prompt

Act as a **Senior QA Automation Engineer** with over **10 years of experience** in Selenium, TestNG, Java, and web application testing. Analyze the **Google Search** functionality and generate a comprehensive set of test scenarios covering **functional, negative, boundary, usability, accessibility, cross-browser, and automation-ready** test cases.

The application under test is the **Google Search homepage**. The objective is to validate that users can search using different types of input and receive accurate, relevant, and reliable search results. Your analysis should cover happy paths, edge cases, error handling, browser compatibility, responsiveness, and performance-related considerations.

Use the following example as a reference:

Scenario: User searches for **"Selenium WebDriver"**

Expected Result:

- Google accepts the search input.
- The Search Results page loads successfully.
- Relevant results related to **Selenium WebDriver** are displayed.
- The entered search keyword remains visible in the search box.
- No unexpected errors or failures occur.

Assume the role of a **Senior QA Engineer** who applies industry-standard testing techniques such as positive testing, negative testing, boundary value analysis, equivalence partitioning, exploratory testing, and risk-based testing to identify both obvious and hidden defects.

Generate the output in the following table format:

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Priority	Test Type	Automation Feasible (Yes/No)

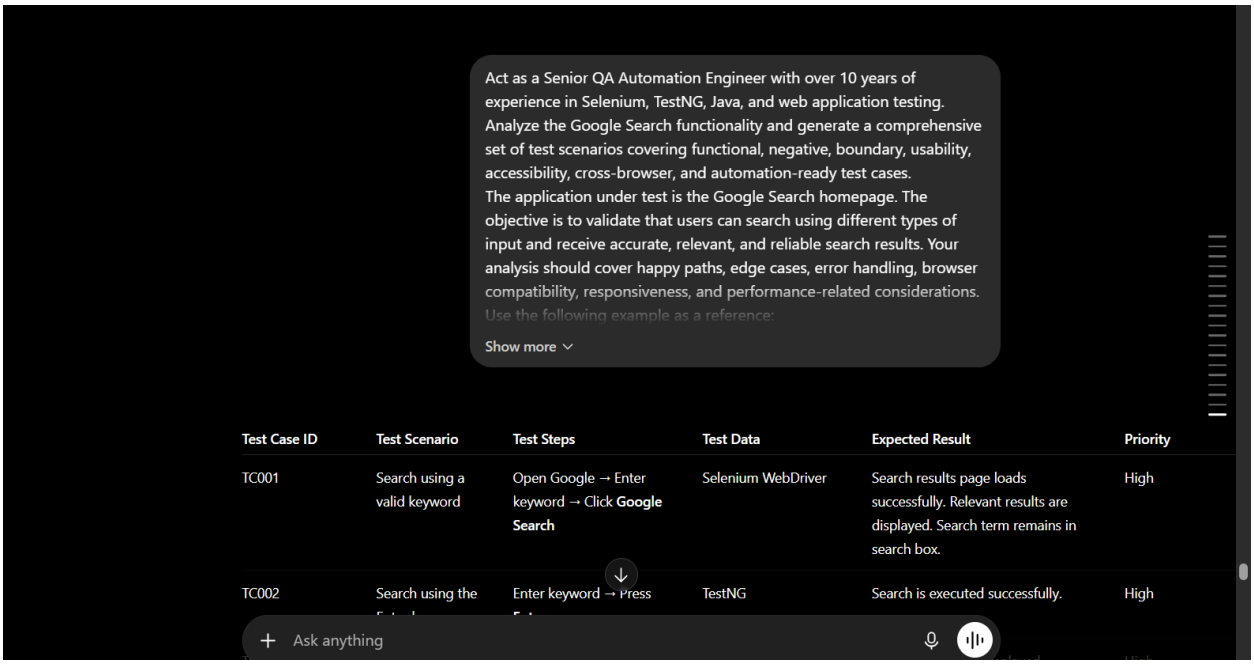
Ensure the generated test suite includes:

- Positive test scenarios
- Negative test scenarios
- Boundary value test cases
- Special character and invalid input test cases
- Cross-browser compatibility scenarios
- Automation feasibility (Yes/No) for each test case

Use a **professional, structured, and QA-focused** tone. Follow industry best practices for manual and Selenium automation testing. Ensure the test scenarios are comprehensive, reusable, and suitable for enterprise-level QA documentation and future automation implementation.

Observations

When I use the week prompt the output was not in a structured format while using ICEPOT components I could able to generate well structured desired output



Scenario Name

File Upload

Weak Prompt

"Generate test cases for file upload."

What I Feel is Missing

What's missing: No file types, no size limits,

no success/failure criteria, no security angle (malicious file upload).

1. Instruction

Act as a Senior QA Automation Engineer. Analyze the File upload functionality and generate comprehensive functional, negative, boundary, usability, and accessibility test scenarios.

2. Context

The application under test is the Google doc attachment. The primary objective is to validate that users can upload different types of files and receive accurate, relevant, and reliable results. The testing should cover happy paths, edge cases, error handling, browser compatibility, responsiveness, and performance-related considerations.

3. Example

For example:

Scenario: User upload the testing documents

Expected Result:

- Google accepts the file.
- Files should loads successfully.
- In case of error , error message should display
- The upload button should be clickable and upload the files.
- No unexpected errors occur.

4. Persona

You are a **Senior QA Engineer** with over 10 years of experience

5. Output Format

Provide the output in a table with the following columns:

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Priority	Test Type

Also include:

- Positive test scenarios
- Negative test scenarios
- Boundary test cases
- Special character test cases
- Cross-browser scenarios
- Automation feasibility (Yes/No)

6. Tone

Use a professional, structured, and QA-focused tone.

✅ Rewritten Prompt

Act as a **Senior QA Automation Engineer** with over **10 years of experience** in Selenium, TestNG, Java, and web application testing. Analyze the **File Upload** functionality for **Google Document Attachment** and generate a comprehensive set of test scenarios covering **functional, negative, boundary, usability, accessibility, cross-browser, performance, and automation-ready** test cases.

The application under test is the **Google Document Attachment** feature. The primary objective is to validate that users can upload different types of files successfully and receive accurate, relevant, and reliable feedback. The testing should cover happy paths, edge cases, error handling, browser compatibility, responsiveness, accessibility, and performance-related considerations.

Use the following example as a reference:

Scenario: User uploads a testing document.

Expected Result:

- Google accepts the selected file.
- The file uploads successfully.
- If an upload fails, an appropriate and meaningful error message is displayed.
- The **Upload** button is enabled, clickable, and initiates the upload successfully.
- No unexpected errors, crashes, or application failures occur.

Assume the role of a **Senior QA Engineer** who applies industry-standard testing techniques such as **positive testing, negative testing, boundary value analysis, risk-based testing** to

identify both functional and non-functional defects. Consider security, usability, accessibility, and browser compatibility while designing the test suite.

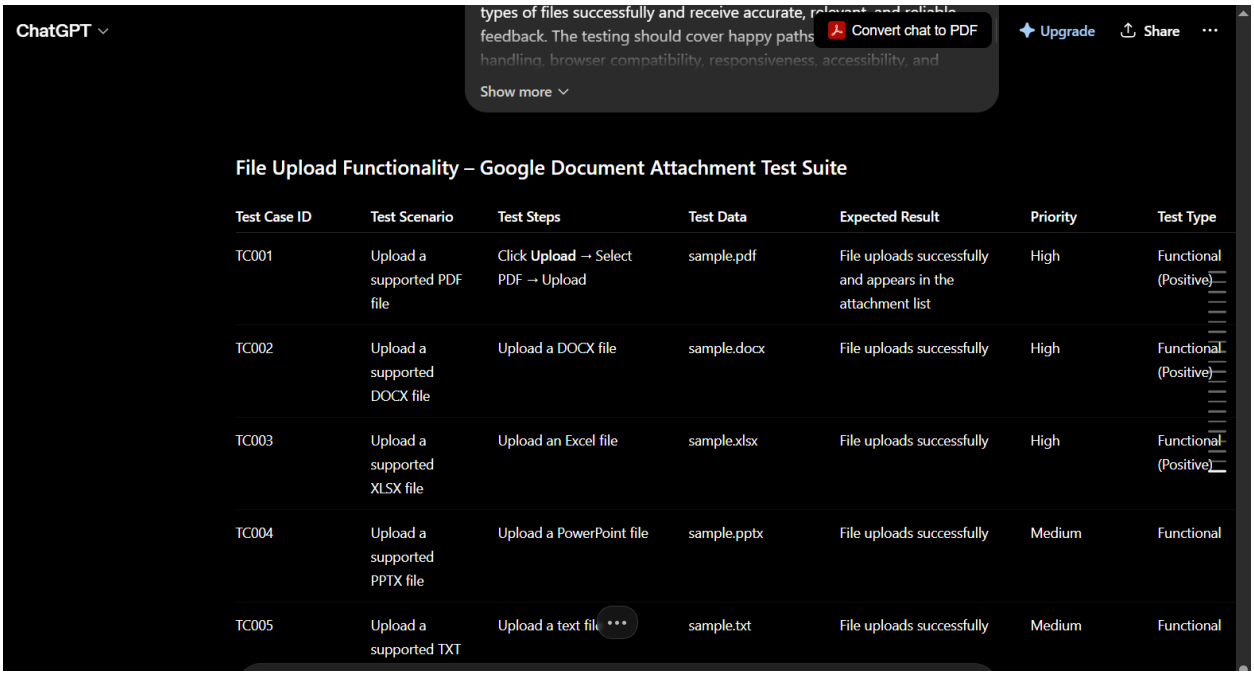
Generate the output in the following table format:

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Priority	Test Type	Automation Feasible (Yes/No)

Ensure the generated test suite includes, but is not limited to, the following scenarios:

- Positive file upload scenarios
- Negative test scenarios
- Boundary value test cases
- Uploading supported file formats (PDF, DOCX, XLSX, PPTX, TXT, PNG, JPG, etc.)
- Maximum allowed file size
- File size exceeding the allowed limit
- Multiple file upload scenarios (if supported)
- Duplicate file upload
- File names containing special characters
- Automation feasibility (**Yes/No**) for every test case

Use a **professional, structured, and QA-focused** tone throughout the response.



Observations

When I use the week prompt the output was not in a structured format while using ICEPOT components I could able to generate well structured desired output

Scenario Name

Role-Based Access Control

Weak Prompt

Write test cases for user roles.

What I Feel is Missing

No role types (Admin, Manager, Viewer),

no permission matrix, no unauthorized access scenarios, no app domain.

1. Instruction

Act as a Senior QA Automation Engineer. Analyze the Role based access control functionality and generate comprehensive functional, negative, boundary, usability, and accessibility test scenarios.

2. Context

The application under test is banking. The primary objective is to validate that users should have appropriate access based on role-based and receive accurate, relevant, and reliable results. The testing should cover happy paths, edge cases, error handling, browser compatibility, responsiveness, and performance-related considerations.

3. Example

For example:

Scenario: User with admin access

Expected Result:

- Should login to the home page with appropriate credentials.
- Verify their employee profile.
- In case of invalid login , error message should display

- login button should be clickable and upload the files.
- No unexpected errors occur.

4. Persona

You are a **Senior QA Engineer** with over 10 years of experience

5. Output Format

Provide the output in a table with the following columns:

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Priority	Test Type

Also include:

- Positive test scenarios
- Negative test scenarios
- Boundary test cases
- No unauthorized access scenarios
- No permission matrix
- No app domain
- Automation feasibility (Yes/No)

6. Tone

Use a professional, structured, and QA-focused tone.

✅ Rewritten Prompt

Act as a **Senior QA Automation Engineer** with over **10 years of experience**. Analyze the **Role-Based Access Control** functionality of a **banking application** and generate a comprehensive set of test scenarios covering **functional, negative, boundary, usability, accessibility, security, cross-browser, performance, and automation-ready** test cases.

The application under test is a **banking application** that implements **Role-Based Access Control (RBAC)**. The primary objective is to validate that authenticated users are granted only the permissions assigned to their roles . The testing should cover happy paths, edge cases, error handling.

Use the following example as a reference:

Scenario: User logs in with **Administrator** credentials.

Expected Result:

- The user successfully logs in with valid credentials.
- The user is redirected to the appropriate home page or dashboard.
- The user's profile information (role, employee details, permissions) is displayed correctly.
- Menus, pages, and actions available to the Administrator are accessible.
- If invalid credentials are entered, a meaningful error message is displayed.
- The **Login** button is enabled and functions correctly.
- No unexpected errors, crashes, or security vulnerabilities occur.

Generate the output in the following table format:

Test Case ID	Test Scenario	Test Steps	Test Data	Expected Result	Priority	Test Type	Automation Feasible (Yes/No)

Ensure the generated test suite includes, but is not limited to, the following scenarios:

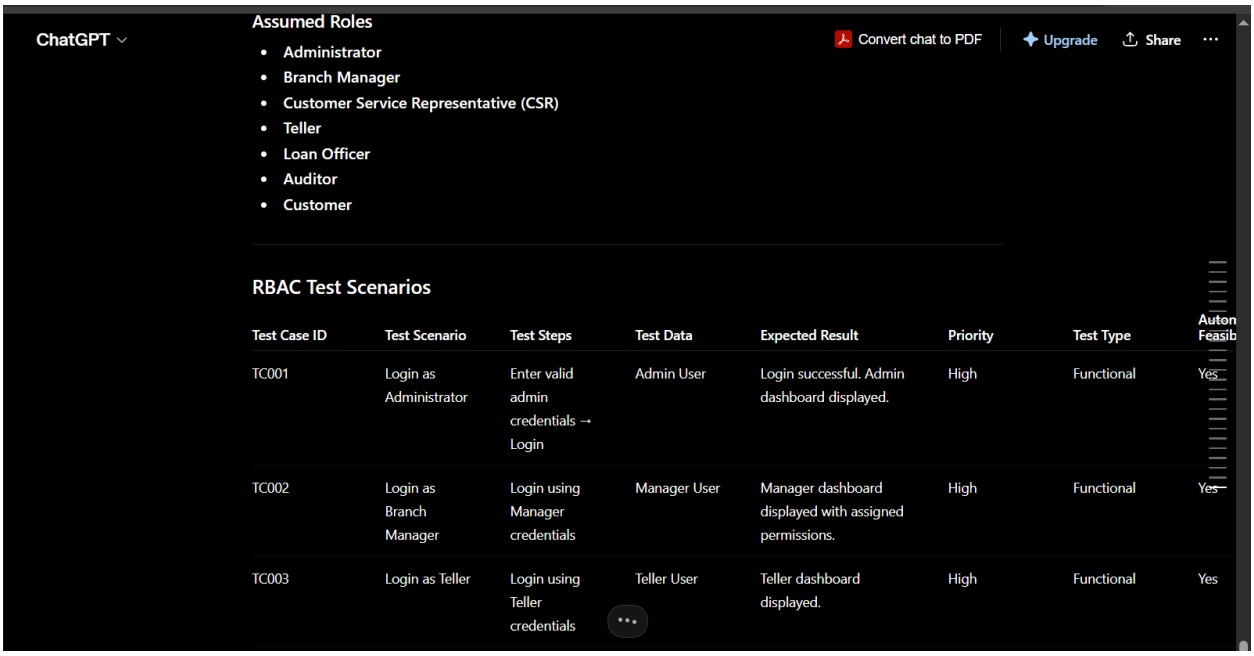
- Positive login scenarios for each supported role
- Negative login scenarios
- Role-based authorization validation
- Unauthorized access validation
- Permission validation for Create, Read, Update, Delete (CRUD) operations
- Access validation for menus, pages, dashboards, reports, and transactions
- Direct URL access validation
- Session timeout and session expiration validation

Additionally, provide the following artifacts:

1. **Role-Permission Matrix** showing which roles can access which modules and actions.
2. **Unauthorized Access Matrix** identifying resources that must not be accessible by each role.
3. **Application Domain Coverage Matrix** mapping application modules (for example: Customer Management, Accounts, Payments, Loans, Reports, Administration, Audit, User Management) to the roles permitted to access them.

For every test case, indicate whether it is suitable for **Selenium/TestNG automation** by specifying **Automation Feasible (Yes/No)**.

Use a **professional, structured, and QA-focused** tone



Observations

When I use the week prompt the output was not in a structured format while using ICEPOT components I could able to generate well structured desired output